Data Management for Machine Learning – Assignment 2

End-to-End Data Management Pipeline for a Recommendation System

Name / ID: Abinaya R | 2025ag05178@wilp.bits-pilani.ac.in
GitHub repository: [ABINAYARAGHUNATH/recommendation-pipeline]

1. Overview and Architecture

This report describes a reproducible, modular pipeline for a movie recommendation system built on the MovieLens 100K dataset (943 users, 1,682 movies, 100,000 ratings). It covers ingestion, validation, preparation, transformation, feature management, model training, evaluation and orchestration. Each stage is a separate Python module that reads its inputs from disk and writes its outputs to disk, all paths and hyperparameters live in one configuration file, and a single Prefect flow runs the stages in dependency order.

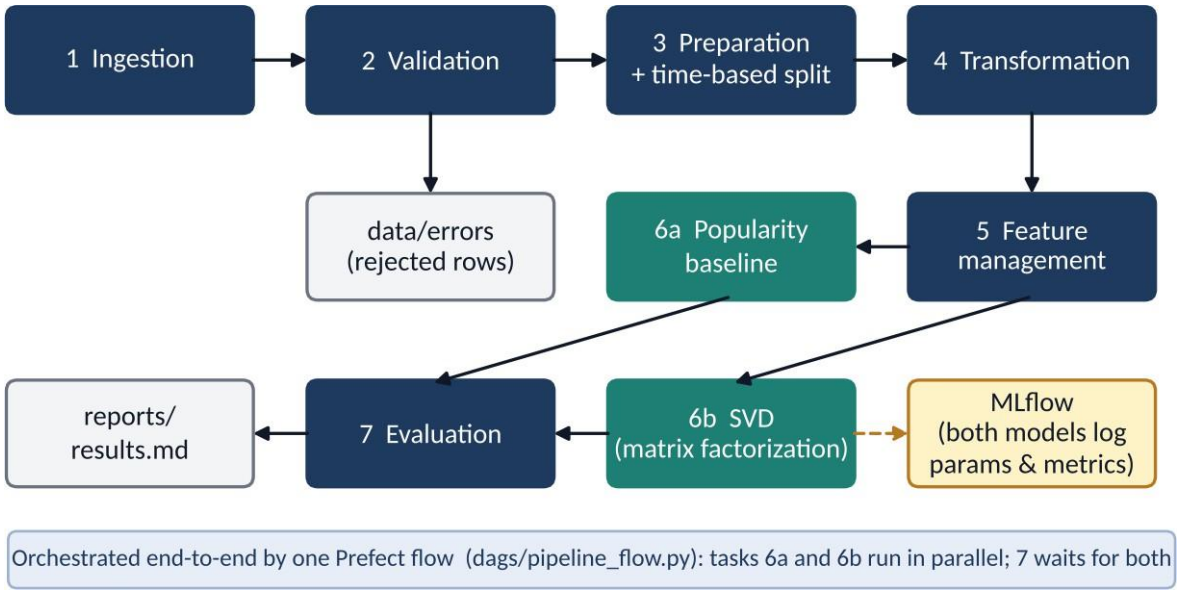


Figure 1. Pipeline architecture. Stages 6a and 6b run in parallel; the dashed arrow is experiment tracking.

2. Pipeline Stages and Design Decisions

Ingestion. Users, items and interactions are loaded and saved unchanged as CSV files in data/raw/. To exercise the validation stage, five rows are sampled, copied and appended (two unchanged, one with its rating set to 9, one with an unknown user ID, one with a negative timestamp), giving 100,005 interactions. The download step is skipped when the data already exists, so re-runs are safe.

Validation. Every interaction is checked for missing values and wrong data types, duplicate user–item pairs, ratings outside 1–5, invalid timestamps (non-positive, unparseable or in the future) and user or item IDs missing from the reference tables. Failing rows are written to data/errors/ with the reason attached instead of being silently dropped, and a summary is saved to reports/validation_report.json (Section 4).

Preparation. Valid rows are de-duplicated and sorted by time, and user and item attributes are standardized (age imputed with the median, gender upper-cased, release dates parsed). The train/test split is time-based, not random: interactions up to the 80th percentile of timestamps form the training set (80,003 rows, ending 7 March 1998) and later ones the test set, so the model never learns from the future. Test rows whose user or item is absent from training are removed to avoid cold-start cases, which shrinks the test set from 19,997 to 2,860 interactions (see Section 6).

Transformation. From the training set only (no leakage): a user activity summary (interaction count, average rating, last activity), item popularity (rating count, average rating) and a sparse user–item rating matrix.

Feature management. User, item and interaction features are produced by one function, build_features(), so the same logic can serve training and prediction. They are documented in a feature catalog (reports/feature_catalog.md): user average rating, activity count, days since last interaction, preferred genre and demographics; item popularity, average rating and genre flags; and interaction rating and recency, all measured relative to the end of the training period.

Training and tracking. Two models are trained: a popularity baseline (the most-rated items the user has not yet seen) and a personalized matrix-factorization recommender (truncated SVD with 20 latent factors on the mean-centred user–item matrix, seed 42). Each run logs its parameters, metrics and model file to MLflow (Figure 3).

Orchestration. dags/pipeline_flow.py defines a Prefect flow of eight tasks with explicit dependencies: ingest → validate * prepare → transform → features → (train popularity ‖ train SVD) → evaluate. A successful run completed in about 5 seconds (Figure 2).

3. Tools Used and Rationale

- **pandas, SciPy, scikit-learn, Parquet:** tabular processing, sparse matrices, SVD, and compact typed storage for intermediate data.
- **Plain pandas validation rules:** few dependencies, and every rule produces an error dataset with a reason per rejected row.
- **MLflow:** records parameters, metrics and artifacts for each run and allows side-by-side comparison.
- **Prefect:** Python-native orchestration with task dependencies, parallel tasks and a monitoring UI; lighter to set up locally than Airflow.
- **Reproducibility:** one configuration file, fixed random seed, idempotent ingestion, file-based hand-off between stages, dependencies listed in requirements.txt, and step-by-step instructions in the README.

4. Data Quality Findings

Validation processed 100,005 interactions: 100,000 passed and 5 were quarantined in data/errors/interactions_errors.csv. The counts below add up to 7 because a row can fail more than one check (the rating-9 and negative-timestamp rows are also duplicate user–item pairs). The faulty rows were injected on purpose to demonstrate the checks.

Check failed	Flags	Meaning
Duplicate user–item pair	4	Same (user_id, item_id) appears more than once; first occurrence kept
Invalid rating	1	Rating outside the allowed range 1–5 (value 9)
Referential integrity (orphan user)	1	user_id not present in the users table (value 99999)
Invalid timestamp	1	Non-positive timestamp (value −5)

5. Models and Results

Evaluation uses K = 10; an item counts as relevant if the user rated it 4 or higher in the test period. RMSE and MAE use the test ratings (the popularity model predicts each item's mean rating). Best value per metric in bold.

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
Popularity (baseline)	0.1105	0.0860	0.1379	0.0458	**0.7486**	**1.0365**	**0.8331**
SVD (personalized)	**0.1158**	**0.1159**	**0.1755**	**0.1510**	0.7242	1.1507	0.9254

- **SVD ranks better:** recall rises from 0.086 to 0.116 and NDCG from 0.138 to 0.176, while precision improves only slightly (0.1105 to 0.1158).
- **Coverage is 3.3 times higher** (15.1% vs 4.6% of the catalogue): SVD recommends a much wider set of movies, whereas the baseline keeps surfacing the same popular titles.
- **Popularity wins on diversity and rating error:** within-list diversity is slightly higher (0.749 vs 0.724) and RMSE/MAE are lower. The error gap is expected because truncated SVD treats unrated cells as zeros, so its scores suit ranking but are not calibrated rating predictions.
- **Caution:** the test set is small (2,860 interactions), so the small precision difference in particular should be read as indicative, not conclusive.

6. Limitations and Future Work

- **Small test set.** MovieLens users rate in short bursts, so many users in the last 20% of the timeline never appear earlier and are removed. A per-user time split (hold out each user's most recent 20%) would give a larger, more representative test set.

- **Features not yet consumed by the models.** The feature tables are generated and catalogued, but the two current models use the interaction matrix and popularity statistics; the features are intended for a hybrid or learning-to-rank model.
- **Model quality.** Truncated SVD does not handle missing ratings; ALS, Surprise or implicit would improve rating prediction. There is no cold-start handling, and evaluation is offline only.
- **Future work:** scheduled Prefect deployments, data-drift monitoring, Pandera or Great Expectations rules, and a feature store such as Feast.

7. Evidence of a Successful Run

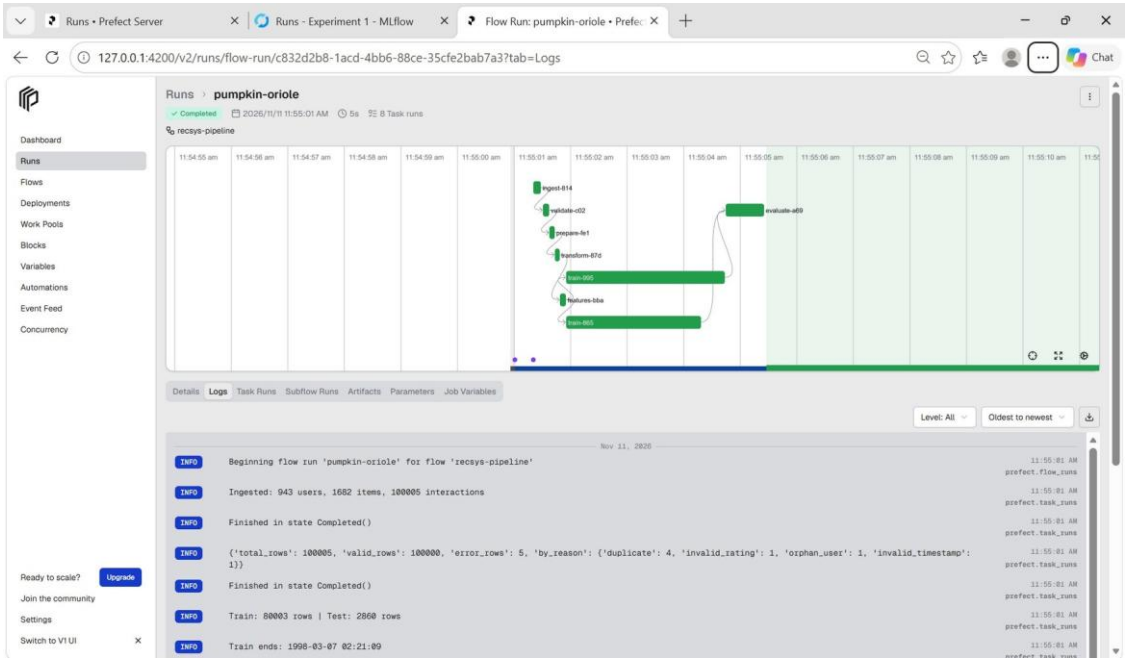


Figure 2. Prefect flow run "pumpkin-oriole": Completed, 8 task runs, about 5 s (task timeline and logs).

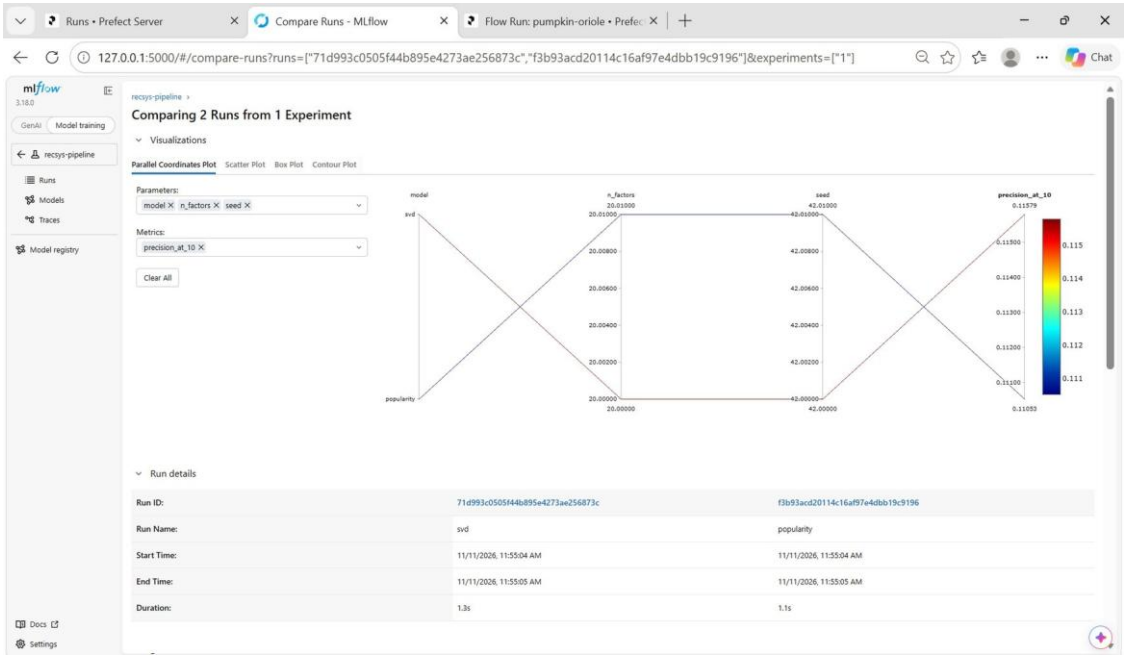


Figure 3. MLflow comparison of the svd and popularity runs (parameters and metrics).